Maharashtra Education Society's
Institute of Management and Career Courses (IMCC), Pune
AUTONOMOUS

Academic Year 2026-27
FY MCA- SEM I
IT12 Data Structure and Algorithms
Practice Assignment1

Roll No: 2601051
Name: Mohit Vijay Dhumal

```
class Node:                 #Creating a Node Class which contains data and pointer to next node
   def __init__(self,data):
      self.data = data
      self.next = None

class SinglyLL:           # Creating Class SinglyLL which conatins Operations on List
   def __init__(self):
      self.head = None

   def createList(self,data):      # method for Appending the List
      new_node = Node(data)
      if self.head == None:
         self.head = new_node
      else:
         temp = self.head
         while(temp.next!=None):
            temp = temp.next
         temp.next = new_node

   def printList(self):          # method for printing the list
      if self.head == None:
         print("List is Empty!")
      else:
         temp = self.head
         while(temp):
            print(temp.data,end=" -> ")
            temp = temp.next
         print("None")

   def insertAtPos(self,data,pos):        # method for inserting a node at particular pposition
         new_node = Node(data)
         if pos<1 or pos>self.countList()+1:
            print("Invalid Position")
```

```
                elif pos==1:
            new_node.next = self.head
            self.head = new_node
        else:
            temp = self.head
            for i in range(pos-2):
                temp = temp.next
            new_node.next = temp.next
            temp.next = new_node

    def middleNode(self):                # method for fining the Middle Node
        if self.head == None:
            print("LinkedList is Empty!")
        else:
            noOfNodes = self.countList()
            middle = noOfNodes//2+1

            temp = self.head
            for i in range(middle-1):
                temp = temp.next
            print("Middle Node:",temp.data)

    def delNode(self,pos):            # method for deleting a Node at Particular Position
        if self.head == None:
            print("LinkedList is Empty!")
        elif pos<1 or pos>self.countList():
            print("Invalid Position!")
        elif pos == 1:
            self.head = self.head.next
        else:
            temp = self.head
            for i in range(pos-2):
                temp = temp.next
            temp.next = temp.next.next

    def reverseList(self):            # method for reversing the List
        temp = self.head
        prev = None
        while(temp):
            nextNode = temp.next
            temp.next = prev
            prev = temp
            temp = nextNode
```

```
                self.head = prev

    def sumConsecutive(self):        # method for summation of consecutive nodes
        if self.head == None or self.head.next == None:
            print("Need Atleast Two Nodes for summation")
        else:
            temp = self.head
            while(temp.next!=None):
                total = temp.data + temp.next.data
                print(total, end=" -> ")
                temp = temp.next
            print("None")

    def countList(self):             # method for counting the nodes
        count = 0
        if self.head == None:
            print("List is Empty!")
        else:
            temp = self.head
            while(temp):
                count+=1
                temp = temp.next
            return count



ob1 = SinglyLL()

ob1.createList(10)
ob1.createList(20)
ob1.createList(30)
ob1.createList(40)
print("Linked List After Creation: ",end="")
ob1.printList()

ob1.insertAtPos(5,1)
ob1.insertAtPos(8,2)
print("After Inserting Node at 1st and 2nd Position: ",end="")
ob1.printList()

ob1.delNode(3)
print("After Deleting the 3rd Node: ",end="")
ob1.printList()
```

```
ob1.middleNode()
print("Sum of two consecutive Nodes: ",end="")
ob1.sumConsecutive()
ob1.reverseList()
print("Reversing the list: ",end="")
ob1.printList()

ob1.reverseList()
print("Printing original List again: ",end="")
ob1.printList()
```

